

Sakhi Bazaar

Technical Interview & Architecture Guide Preparing for Platform Review

Overview

This preparation guide addresses the technical architecture, custom logic, and developmental frameworks implemented across the Sakhi Bazaar platform. It includes folder structures, authentication mechanisms, token-based verification flows, database entities, Gemini GenAI description workflows, local pricing aggregation pipelines, role-based authorization protocols, and production deployment schemes.

Subject:	Full-Stack System Interview
Architecture:	MERN (MongoDB, Express, React, Node.js)
Date:	July 2026

Q1: What is your folder structure?

The project is structured as a monorepo containing distinct applications for the main marketplace and the administration portal:

Key Workspaces

- backend/ - Main Node.js/Express server defining database schemas (models), request control rules (controllers), middleware validators, routing files, and app start logic (server.js).
- frontend/ - Main client storefront SPA designed using React 19, Tailwind CSS v4, and Vite. Includes UI views, custom state hook systems, and language/theme context hooks.
- admin/ - Standalone administrative system. Houses 'admin/backend' (Express API server mapping platform-level analytical metrics) and 'admin/frontend' (dashboard interface targeting platform vetting).

Directory Architecture:

```
SakhiBazaar_New/
|-- backend/
|   |-- config/           # DB & socket initializations
|   |-- controllers/      # Express handlers (Auth, Product, AI)
|   |-- models/          # MongoDB schemas
|   |-- routes/          # API routes definitions
|   |-- server.js         # Server startup script
|-- frontend/            # React / Tailwind Storefront
|   |-- src/components/   # UI elements (Navbar, Cards, Filters)
|   |-- src/pages/       # Main views (AddProduct, Cart, Chat)
|   |-- vite.config.js    # Vite compilation configurations
|-- admin/               # Standalone Administration Area
|   |-- backend/         # Express admin server
|   |-- frontend/       # React admin panel client
|-- srs.md               # Software Requirements Specs
```

Q2: How does authentication work?

The platform implements standard credentials login and Google federated Single Sign-In (SSO) securely verified via backend token evaluation:

1. Registration and Hash Storage

- Sign Up (POST /api/auth/register) evaluates strict verification constraints. Phone numbers must contain exactly 10 numeric digits, and Aadhaar numbers must contain exactly 12 numeric digits.
- Passwords must satisfy strength validations (8+ chars, 1 uppercase, 1 lowercase, 1 number, 1 special character).
- Upon successful format validation, the plaintext password is cryptographically salted and hashed using bcryptjs. This hash occurs automatically inside the Mongoose pre-save middleware in User.js before being written to MongoDB.

2. Authentication Actions

- Credentials Login (POST /api/auth/login) checks whether the user enters a matching Email or Username, then verifies the password hash using bcryptjs.
- Google Social Authentication (POST /api/auth/google-login) takes Firebase authenticated token credentials. If the email doesn't exist, it registers a user profile automatically using a strong random password.
- OTP Resets (POST /api/auth/forgot-password) generates a 6-digit numeric verification OTP, registers it in MongoDB, and expires it in 10 minutes. Users reset the password securely after OTP confirmation.

3. Authorization Middleware

- After authentication, a JWT is signed with the user's ID. The client attaches this to the header of subsequent requests: Authorization: Bearer <token>. The protect middleware parses, decodes, and populates the user object on req.user.

Q3: Which JWT flow do you use?

The application utilizes a stateless, token-based Bearer Token flow handled through standard HTTP request headers:

1. Generation Step

- When a user logs in, registers, or authenticates via Google login, the server triggers `generateToken(user._id)`.
- This generates a JWT signed with a secret key (`JWT_SECRET`) with a 30-day expiration period, containing the user's MongoDB ID as the payload.

2. Storage Step

- The server sends the token as part of the JSON response payload. The client intercepts the response and persists the JWT in `localStorage` or in-memory React context state.

3. Request and Validation Step

- For all protected operations, the client appends the token to the HTTP request headers: `Authorization: Bearer <token>`.
- The server's `protect` middleware extracts this header, verifies the token's validity, and fetches the associated user from the database. Since sessions are stored in the token itself, the backend remains stateless and highly scalable.

Q4: Which APIs have you created?

The platform includes two API servers running concurrently to partition buyer/seller operations from admin moderation tasks:

1. Main REST API Router (/api/ prefix)

- Auth & Profiles (/api/auth) - Post register, login, google-login, logout, get/update user profile, change system preferences (theme, language), and trigger OTP forgot-password flows.
- Products Catalog (/api/products) - Create listings (Sellers only; handles uploads), edit details, delete items, list active products, search by keywords, filter attributes, and post product reviews.
- Generative AI Assist (/api/ai) - Generate storytelling product descriptions and WhatsApp/Instagram captions using Google Gemini API.
- Market Metrics (/api/market) - Pull commodity raw materials benchmarks and market trends statistics.
- Cart & Transactions (/api/cart, /api/orders, /api/payments) - Manage shopping cart items, record purchases, create payment intents (Stripe), and track shipment progress.
- Interactions (/api/chat, /api/wishlist, /api/bookmarks, /api/reviews) - Handles real-time messaging, wishlists, bookmarks, and user reviews.

2. Standalone Admin REST API (/api/admin/ prefix)

- Auth (POST /auth/register, /auth/login) - Admin credentials management.
- Seller & Listing Vetting - Approve/suspend seller profiles, flag inappropriate products, list transactions, and compile analytics (total users, listings, sales volume, platform commission).

Q5: Which MongoDB collections do you have?

The platform's database structure uses Mongoose schemas to represent the following collections:

- users: User records (name, email, password hash, role, approval status, preferences).
- products: Listing details (title, category, price, images, SKU, stock quantity), seller reference, and embedded reviews schema.
- carts: Current shopping carts, containing product references, quantities, and saved items.
- categories: Standardized categories taxonomy (e.g. Clothing, Handmade, Food).
- conversations: Messaging threads between buyer and seller accounts.
- messages: Chat logs containing message text, sender, and conversation link.
- notifications: User alerts for orders, chat messages, and portal approvals.
- orders: Purchase details, customer reference, items, pricing, and fulfillment state.
- payments: Auditing logs for Stripe/Razorpay transactions.
- shipments: Courier delivery updates, tracking codes, and recipient links.
- bookmarks: User bookmarked selections.
- marketprices: Seeded commodities pricing data (Pashmina wool, wild honey, clay).
- marketrends: Category demand indicators and popular products lists.

Q6: How exactly does Gemini generate descriptions?

Generative AI descriptions are managed by the Express aiController backend using the Google Gemini SDK:

1. Generation Triggers

- Sellers request description generation by sending product title, category, and optional keywords to POST `/api/ai/generate-description`.

2. Context and Prompting

- The server builds a prompt specifying persona, context, and formatting rules:

You are an expert marketing copywriter for 'Sakhi Bazaar', an online marketplace that empowers women entrepreneurs. Write a highly compelling, professional, and warm product description for a product with the following details:

- Title: [Product Title]
- Category: [Category]
- [Optional Keywords]

Rules:

1. Write in a warm, professional, storytelling style.
2. Keep the description between 80 to 120 words.
3. Return ONLY the plain text description itself. No intro, labels, or markdown.

3. Robustness and Model Fallbacks

- The request runs through a retry utility `callWithRetry` that automatically switches between compatible fallback models (`gemini-2.5-flash`, `gemini-3.5-flash`, `gemini-2.0-flash-lite`, `gemini-2.0-flash`) in case of rate limits (HTTP 429). It uses exponential backoff to handle temporary service disruptions, returning the finalized text to the UI.

Q7: How does market price analysis work?

The platform computes and displays market pricing data on two layers to keep pricing transparent and competitive:

1. Real-Time Platform Aggregations (Local Pricing)

- The seller dashboard queries `GET /api/products/stats/category?category=<category>` to calculate price metrics based on active listings in that category.
- The backend runs a Mongoose aggregation pipeline:

```
const stats = await Product.aggregate([
  { $match: { category: { $regex: `^${category}$`, $options: 'i' }, status: 'active' } },
  {
    $group: {
      _id: '$category',
      avgPrice: { $avg: '$price' },
      minPrice: { $min: '$price' },
      maxPrice: { $max: '$price' },
      count: { $sum: 1 }
    }
  }
]);
```

- This returns the calculated average, minimum, and maximum prices, helping sellers price new items competitively.

2. Commodity Benchmarks

- The endpoints `/api/market-prices` and `/api/market-trends` return seeded data for raw materials (such as Pashmina Wool, Mulberry Silk Yarn, Terracotta Clay) to give sellers visibility into regional wholesale prices.

Q8: How is role-based access implemented (Admin/Seller/Buyer)?

Role-based access control (RBAC) is implemented across the database, routing middleware, and frontend views:

1. Schema Definitions

- The User model defines role ('customer', 'seller', 'admin') and status ('approved', 'pending', 'suspended') fields.
- Sellers register with a status of 'pending' and must be approved by an administrator before they can list products.

2. Router Protection Middleware

- Specific middlewares guard Express controllers on the backend:

```
// Verifies the user is logged in (populates req.user)
const protect = async (req, res, next) => { ... next(); };

// Restricts access to sellers only
const seller = (req, res, next) => {
  if (req.user && req.user.role === 'seller') { next(); }
  else { res.status(403).json({ message: 'Seller role required.' }); }
};
```

- Routes are guarded by chaining these middlewares (e.g. router.post('/', protect, seller, createProduct)).

3. Frontend UI and Route Guards

- React dashboards are customized by role (SellerDashboard, CustomerDashboard, AdminDashboard). Frontend routes are wrapped in a ProtectedRoute component that redirects users if their active role doesn't match the route permissions.

Q9: How did you make the application responsive?

The client application is styled with Tailwind CSS (v4), using utility classes and responsive breakpoints to adapt the layout to any screen size:

1. Responsive Grids

- Product listing grids scale dynamically based on screen width breakpoints: `grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4`. This renders 1 column on mobile, 2 on tablets, and 4 on desktop screens.

2. Direction Scaling

- Layout elements stack vertically on mobile screens and align horizontally on larger screens: `flex flex-col md:flex-row`.

3. Fluid Paddings & Heights

- Containers use mobile-first responsive padding classes (e.g. `px-4 sm:px-6 lg:px-8`) and set fluid heights to prevent content overflow on smaller devices.

4. Conditional Visibility

- Utility classes like `hidden sm:flex` or `hidden lg:block` hide non-essential elements on mobile screens and show them on desktop viewports, keeping the mobile interface clean.

Q10: How would you deploy it?

The application is designed for cloud-based deployment, separating static assets, backend APIs, and database hosting:

1. Frontend SPA Deployment

- Compile React assets using `npm run build`. Deploy the output 'dist/' folder to a CDN-backed hosting platform like Vercel, Netlify, or AWS S3 + CloudFront. Configure routing to redirect all subroutes to `index.html` to support React Router client-side routing.

2. Backend Server Deployment

- Deploy backend code to cloud application hosts like Render, Railway, AWS Elastic Beanstalk, or containerized within Google Cloud Run. Use PM2 on virtual machines for process monitoring, memory management, and automatic crash restarts.

3. Database Hosting

- Host database instances on MongoDB Atlas (managed database-as-a-service), using IP whitelisting to restrict access only to the backend server's IP address.

4. Media & Cloud Services

- Upload product images to Cloudinary (configured via backend middleware) to optimize images and serve them through their global CDN. Use production Google AI Studio keys for Gemini API requests.

5. Environment Secrets and CI/CD

- Store sensitive credentials (`MONGO_URI`, `JWT_SECRET`, `GEMINI_API_KEY`, etc.) securely as environment variables on the hosting platform. Set up a CI/CD pipeline using GitHub Actions to automate testing and deployment on push to the main branch.